

PseudoDOM-Guard, Unified

A Reconciled Architecture for Declarative and Inferred UGC Region Protection

Reconciliation Specification

Merging four independent design efforts into one architecture

Prepared 2026-08-04

Four sources reconciled here

A — the original chat-designed PseudoDOM concept (declarative, developer-integrated)

B — PseudoDOM_Architecture.docx, the first refinement pass

C — "PseudoDOM Guard," USENIX submission draft (2026-03-29)

D — the "PseudoDOM-Guard" project notes 001–009 (2026-03-30, supersedes C where they conflict)

Table of Contents

Table of Contents	2
0. Purpose and Provenance	4
1. Background	4
1.1 A Direct Connection Worth Stating Plainly	4
2. The Two Deployment Models	5
2.1 Declarative (A/B) — the Developer Cooperates	5
2.2 Inferred (C/D) — Nobody Asked the Developer Anything.....	5
2.3 Why They Compose Rather Than Compete	5
3. Unified Layer Architecture	6
L0 — Trust Signal Sources	6
L1 — Early Installation	6
L2 — Unified Sink Interposition	6
L3 — Dual Analysis Substrate.....	7
L4 — Context-Sensitive Inert Parsing.....	7
L5 — UGC Region Localization	7
Candidate generation and pruning	8
Feature vector and model.....	8
Thresholds and declarative override	8
L6 — Two-Stage Mutation Gate.....	8
Stage A — universal bootstrap precheck.....	8
Stage B — selective UGC-aware scrutiny.....	8
Ordering rule carried over from B, unchanged.....	9
L7 — Promotion and Optional Hardening	9
4. Threat Model and Bypass Taxonomy	10
5. Deployment Honesty: Claims Made and Not Made	12
5.1 Claims This Architecture Supports	12
5.2 Claims This Architecture Explicitly Does Not Make	12
6. Ethics and Privacy	12
7. Consolidated Comparison Table	13
8. Data Flow Specification for the Architecture Diagram	14
9. Provenance Map	14
10. Summary and Recommendation	16
10.1 Open Items Before Implementation	16

Appendix A — Unified Sink Coverage (L2) 17

Appendix B — UGC Feature Families (L5) 17

Appendix C — Purification Checklist Additions 18

0. Purpose and Provenance

Four separate design efforts converge in this document, produced at different times, by different methods, aimed at overlapping but not identical versions of the same problem: browser-side protection against stored and DOM-based XSS delivered through user-generated-content (UGC) rendering paths.

Source	What it is	Core assumption
A — chat design	A wrapper-interception concept developed in conversation: intercept DOM mutation APIs, stage content in a detached template, sanitize before promotion, batch for performance	Developer integrates the library deliberately
B — first refinement (docx)	Formalized A into a two-tier call model, a seven-stage pipeline, host-config hooking, an attribute/URL injection lane, and a Trusted Types pairing recommendation	Same as A, refined for framework (React/Vue) deployment
C — USENIX draft, 2026-03-29	A five-layer browser-extension architecture: network interception, pre-execution wrapper install, PseudoDOM + 63 wrapped sinks, a learned UGC classifier, a single mutation gate	No developer cooperation; the extension protects arbitrary sites
D — project notes, 2026-03-30	A revision of C, six layers, tightened for peer review: two deployment modes, a two-stage bootstrap+selective gate, context-sensitive inert parsing, explicit candidate generation/pruning, a bypass taxonomy, ethics/privacy scope	Same as C, with claims narrowed to what is actually defensible

D revises C directly — same system name, one day later, with a "claims we should not make" section that walks back several of C's stronger assertions. Where C and D disagree, this document follows D. A and B remain valid for a different deployment model D and C never targeted: a site whose developers deliberately adopt the protection rather than an extension imposing it on an uncooperative site. The point of this document is that A/B and C/D are not competing answers to the same question — they answer two different versions of the question, and a combined architecture can use both.

1. Background

Carried forward from B, stated once here rather than four times: tree construction (parsing) and DOM graph mutation are different code paths. `appendChild`, `insertBefore`, and `moveBefore` splice existing nodes directly and never invoke the tokenizer; only APIs that hand the browser a raw string (`innerHTML`, `insertAdjacentHTML`, `document.write`, `Range.createContextualFragment`, `DOMParser`) trigger a parse. This is why a sink taxonomy, not a blanket "watch everything" policy, is tractable at all — both C and D organize their entire wrapper surface around exactly this distinction.

1.1 A Direct Connection Worth Stating Plainly

D's revision from a generic detached-document parse to context-sensitive inert parsing — using an inert template for generic fragments, `Range#createContextualFragment` when destination context matters, and specialized containers for table- and namespace-sensitive content — is, independently arrived at, the same lesson as fragment parsing's context-tag-name parameter: a table-destined fragment parsed outside a table context produces a different tree than the same string parsed with the correct context,

because tree construction is context-sensitive by specification, not merely by convention. B's original Stage 2 (parse into one generic detached <template>) had the same blind spot C originally had, and D's fix is the correct one for both.

2. The Two Deployment Models

The reconciliation hinges on naming this distinction precisely, because it explains nearly every difference between A/B and C/D rather than treating them as competing designs.

2.1 Declarative (A/B) — the Developer Cooperates

A library or framework hook a development team deliberately integrates. The application knows exactly which containers render database-sourced UGC, because it wrote the code that fetches and renders it. Trust zones are asserted, not inferred: a data attribute, a config entry, or a registered API call marks a container as untrusted, with zero classification risk, because the ground truth is simply known.

2.2 Inferred (C/D) — Nobody Asked the Developer Anything

A browser extension, installed by the end user, protecting sites that did nothing to earn or request that protection. There is no channel to the application's own knowledge of where UGC lives, so "which subgraph is the comment region" has to be discovered structurally — hence D's candidate generation (wildcard-XPath sibling-family grouping), pruning, and a logistic regression classifier scoring a nine-family feature vector. This is real, unavoidable model risk (precision/recall on a discovery problem) that the declarative model simply does not have, because it never has to guess.

2.3 Why They Compose Rather Than Compete

Both branches ultimately populate the same thing: a per-node region-confidence map that a mutation gate consults before deciding how much scrutiny an insertion gets. The declarative branch populates that map with certainty; the inferred branch populates it with a score. Nothing about the gate downstream needs to know which branch supplied the signal — it only needs a confidence value. A site that adopts the declarative library, viewed through a browser that also runs the extension, gets both signals simultaneously: the extension's classifier will very likely independently score the declared containers as high-confidence UGC too (real comment sections structurally resemble what the classifier was trained to find), and where a declared zone exists, it should simply outrank an inferred one rather than requiring agreement. This is the single architectural decision that turns four separate documents into one system: §3–6 below build that shared map and the gate that consumes it.

3. Unified Layer Architecture

Eight layers, each attributed to the source(s) it draws from, reconciling B's tiered call model, C's five layers, and D's six layers into one pipeline.

L0 — Trust Signal Sources

Two independent producers feed the same region-confidence map:

- **Declarative signal (A/B):** a developer-asserted zone tag, carrying maximum confidence, no scoring needed.
- **Page-level priors (C/D, Layer 1 / §4.2 of C):** extracted from HTTP response headers before any HTML parses — CSP posture, Open Graph type, JSON-LD comment/review markers, framework fingerprint, comment-route hints in embedded script text. These do not classify a region by themselves; they bias the Layer-5 classifier's prior.

L1 — Early Installation

Two paths, matching D's explicit deployment-mode split (§003), because "how early can this install" has a different, honestly-reported answer in each:

Path	Mechanism	Deployment mode	Guarantee
Framework hook (A/B)	React host config / Vue custom renderer, or a bundled library initializing before app code runs	Declarative, developer-integrated	Strong within the framework's own render path; no coverage outside it
Content script, document_start, world:MAIN	Chrome Manifest V3 content script	Extension, always-on	Primary deployable path; empirically measured, not assumed (D, §004)
CDP Page.addScriptToEvaluateOnNewDocument	Debugger-backed injection	Extension, measurement mode only	Stronger ordering guarantee; requires active chrome.debugger attachment, not part of the always-on story

D's explicit correction to C matters here: C treated pre-execution timing as effectively guaranteed; D treats it as an empirical question with a measured success rate and a named failure taxonomy (the "stolen reference" bypass — a page script that runs first and caches the native function before the wrapper installs). Both A/B and C/D inherit this same threat: even the declarative library is just another script subject to bundle load order. This is precisely the bypass class Trusted Types (L7) closes and a wrapper alone cannot, regardless of deployment model.

L2 — Unified Sink Interposition

B originally split calls into a "Tier 1, no purification needed" graph-mutation class and a "Tier 2" string-sink class, and left a real gap: a live document.createElement('script') plus appendChild, or a JS-property-set.onerror, touches neither a string sink nor triggers a parse, and slipped past that split entirely. C and D close this by construction — both wrap the full mutation-API surface (node methods, element methods, property setters via descriptor patching, document-level APIs, attachShadow) at the prototype level, node-construction and property-setter calls included, so nothing distinguishes "was this a string" from "was this a live node" at the interposition layer itself; every mutation-shaped call reaches the same gate. This document adopts C/D's full-coverage model and retires B's tier split — Appendix A carries the merged sink list, with priority markers (P0/P1/P2) preserved from C for implementation sequencing, not for security exemption.

L3 — Dual Analysis Substrate

B and C/D built two structurally different things that both got called "the staging area," and they are not substitutes for each other — they answer different questions and both are needed.

- **The PseudoDOM (C/D):** a lightweight, WeakMap-identified structural log/mirror, built incrementally from every intercepted call's metadata (tag, attributes, children, depth, mutation type, timestamp), never touching the live DOM or triggering reflow. It answers "where in the tree structure is this node, and what is its ancestor chain's confidence" — a fast lookup structure for L5/L6, not a place content is actually parsed.
- **The detached staging parse (A/B):** the actual, real parse of a string-sink's content into an inert target, run once so the security pipeline can inspect the real resulting tree — including anything foster parenting or the adoption agency algorithm relocated — before it reaches the live document. It answers "what does this specific string actually produce, structurally," which the PseudoDOM's log does not attempt to answer on its own.

Combined: L2 records into the PseudoDOM unconditionally (cheap, always happens); the detached staging parse (L4) runs only for string-sink calls whose target the region map (L5) flags for scrutiny, keeping the expensive step selective exactly as both C's and D's architectural invariant requires.

L4 — Context-Sensitive Inert Parsing

D's corrected design, replacing B's and C's original generic-<template> approach, per §1.1 above:

- Generic HTML fragments: an inert <template>.content parse.
- Fragments whose destination context matters (table-family content, foreign-namespace content): Range#createContextualFragment, or a specialized container matching the real destination context.
- Never attach the resulting fragment to the live document before the gate (L6) renders a verdict.

The purification step from B (DOMPurify, IN_PLACE mode, no round-trip through a second string — see B §3.2 Stage 4) runs directly against whichever of these produced the fragment, unchanged by this correction.

L5 — UGC Region Localization

D's explicit split of candidate generation from candidate scoring, which B never had reason to build (declarative zones need no discovery step) and C treated too casually (D's own critique: "the model cannot rank what the system never proposes").

Candidate generation and pruning

- Propose: repeated sibling families above a minimum cardinality; ancestors of semantic anchors (comment/reply/review markers); composer-adjacent containers; newly emerged repeated structures after SPA route transitions.
- Prune: remove nodes under navigation/header/footer ancestry; suppress commerce-dominant or table-dominant patterns; collapse near-duplicate ancestors with nearly identical feature vectors; cap scored candidates per page.

Feature vector and model

Nine families (structural repetition, semantic keywords, author/avatar/profile coverage, time signals, per-unit actions, composer proximity, text statistics, negative controls, page-global priors from L0) scored by a regularized logistic regression, chosen for sub-millisecond in-browser inference, direct interpretability for ablation, and better small-dataset generalization than a boosted-tree or deep model — the justification is unchanged across C and D and is sound independent of either paper's specific dataset. Appendix B carries the full feature family list.

Thresholds and declarative override

Three region states, not two, per D: unknown, low-confidence, high-confidence, governed by two calibrated thresholds (T_{low} , T_{high}) tuned on a held-out domain split rather than fixed heuristically. A declarative zone tag from L0 short-circuits this scoring entirely and is treated as unconditionally high-confidence — inference exists only for containers nobody labeled.

L6 — Two-Stage Mutation Gate

D's single most valuable addition, closing a gap neither A/B nor C had an answer for: on an SPA page, the mutation that creates the comment subtree can fire before L5 has scored anything at all. A design that only gates high-confidence regions leaves that first, often decisive, write completely ungarded.

Stage A — universal bootstrap precheck

Applied to every mutation reaching a fixed, short list of definitionally high-risk sink writes, regardless of region-map state: string-to-HTML sinks, event-handler attribute writes, javascript: URL assignments, script-node construction and activation, document.write. Cheap and deliberately conservative — it exists to catch the obvious payload shapes before classification has had any chance to run, not to replace full scrutiny.

Stage B — selective UGC-aware scrutiny

Once L5's region map has a score for the target: high-confidence targets receive the full precheck (L4's parse plus purification plus B's structural-fidelity check, comparing actual post-parse ancestry against what the caller's intent implied); low-confidence targets receive lighter inspection; everything else passes through at the cost of a WeakMap lookup and a bounds check, matching D's stated architectural invariant almost verbatim.

Ordering rule carried over from B, unchanged

Within a single flush cycle, aggregate every pending operation before running purification once over the assembled result — never purify each call individually and batch the already-purified pieces. Sanitizing per-call and batching afterward reopens exactly the fusion-across-boundaries bypass this course's own text-fusion material (Module 09 / Track 2 Module T2-03) demonstrated: content that looks inert in isolated fragments can combine into something that is not once assembled and reparsed together.

L7 — Promotion and Optional Hardening

Verdicts from L6 resolve to pass, sanitize, block, or log; a passing or sanitized result is promoted using pure graph operations only (`appendChild` / `moveBefore`), which per §1 cannot themselves trigger a parse and therefore cannot reopen anything L4–L6 already resolved.

Trusted Types is positioned, per D's §007 (a direct, independently-reached match to B's §3.6), as optional downstream hardening rather than a core dependency: where a CSP directive enforces it, sanitized output is handed to sinks as a registered policy's `TrustedHTML` rather than a raw string, closing the "stolen reference" bypass class for any code path a wrapper alone cannot see — but it adds nothing beyond wrapper-equivalent risk when the enforcing directive is absent, and neither document claims otherwise.

4. Threat Model and Bypass Taxonomy

Merged from B's Word-doc gap analysis, C's "adversary can/cannot" model, and D's explicit Bypass 1–7 list (§002) — the fullest single account across all four sources.

Bypass	Source	Mitigation in this architecture	Residual risk
Stolen reference: page script caches the native sink before wrapper install	B, D (Bypass 1)	Earliest-available install path (L1) plus Trusted Types (L7) as a platform-enforced fallback	Still open for any code that runs before both, on sites without CSP enforcement
Restore/replace patched prototypes post-install	D (Bypass 2)	Non-reconfigurable descriptors where supported; integrity checks; tamper logging	Fully malicious origin remains explicitly out of scope in both C and D
Fresh iframe/OOPIF context missed by injection	B, D (Bypass 3)	Per-frame injection where platform permits; declared as a measured coverage gap, not assumed solved	Cross-origin OOPIF coverage is platform-dependent and must be measured, not assumed
Bootstrap gap: first UGC-creating mutation precedes classification	D (Bypass 4, Challenge 3)	L6 Stage A universal bootstrap precheck	Bootstrap policy is intentionally conservative, not a substitute for full scrutiny
document.write / writeIn parser-driven insertion	D (Bypass 5)	Wrapped identically to other string sinks (L2)	None beyond the general early-install risk
Content hidden inside Shadow DOM	D (Bypass 6)	attachShadow wrapped; shadow roots represented explicitly in the PseudoDOM (L3)	Depends on shadow-root mode coverage; closed-mode edge cases need measurement
Parser-context mismatch for string sinks (e.g., table-destined content)	D (Bypass 7)	Context-sensitive inert parsing (L4)	Not every sink context has a clean specialized-container mapping; documented as a known limit, not silently dropped
Live-node attachment bypassing string-sink checks entirely	B (identified gap)	Full prototype-level coverage of node/element construction methods (L2), not just string sinks	Closed by C/D's architecture; B alone did not close this
DOM clobbering via id/name collision with global identifiers	C (Table 8 evaluation)	Added to the purification checklist (Appendix C) — requires SANITIZE_NAMED_PROPS-equivalent handling, not default in every sanitizer config	Easy to omit if a purification config is copied without this flag
Attribute/URL-scheme injection through ordinary property bindings (href, src, javascript:)	B §3.4	Dedicated Tier-1-equivalent scheme-allowlist lane, independent of the parse-driven pipeline	Requires the lane to be applied uniformly to every dangerous attribute name, not just string-sink content

Bypass	Source	Mitigation in this architecture	Residual risk
Fusion-across-boundaries: per-call sanitization then batching	B (Stage 6 ordering)	Aggregate before purifying once per flush cycle (L6)	Requires the ledger to genuinely defer purification, not just defer promotion

5. Deployment Honesty: Claims Made and Not Made

Carried directly from D §009, generalized to cover both deployment models rather than only the extension.

5.1 Claims This Architecture Supports

- Interposition at the earliest point each deployment model actually allows — measured for the extension path, structurally guaranteed within a framework's own render path for the declarative path.
- Selective, UGC-focused scrutiny that keeps overhead concentrated on the fraction of mutations that matter, validated against a domain-diverse corpus for the inferred branch and trivially true for the declarative branch (the developer already knows which containers to scrutinize).
- Defense-in-depth for a specific, well-scoped XSS delivery pathway — stored and DOM XSS reaching a DOM sink through UGC rendering — not a claim to eliminate XSS as a category.

5.2 Claims This Architecture Explicitly Does Not Make

"We intercept all DOM writes before any page code, on every site, unconditionally." — "This system is robust against a fully malicious, actively anti-extension origin." — "Trusted Types closes every remaining bypass." — "A single detached parse perfectly models every sink's real parsing context." — "This eliminates XSS."

Each of these was an overclaim identified in D's own reviewer-risk register (§009 §3–4) and is inherited here as a standing limitation, not a solved problem.

6. Ethics and Privacy

Applies wherever this architecture is operated as a research artifact or a production extension collecting telemetry; carried from D §009 §6 and §003 §10.

- No live XSS payloads are submitted to third-party production sites without explicit authorization; end-to-end protection evaluation uses local, controlled, or explicitly authorized test environments only.
- Two operating modes with different data-retention policies: a data-collection mode (feature vectors, screenshots, richer mutation metadata, for labeling and research) and a protection mode (minimal, security-relevant logs only, no persisted raw UGC text by default).
- Where content must be persisted for labeling, prefer structural features, hashed text, or short redacted previews over full raw bodies; human reviewers label structural candidate regions, not arbitrary private page content.

7. Consolidated Comparison Table

Mechanism	Deployment model	Region signal	Bootstrap-safe?	Bypass-resistant?	Sink coverage
A/B alone (declarative wrapper)	Developer-integrated	Asserted	N/A — zones known upfront	No (no CSP pairing)	String sinks + graph mutation (after B's revision)
A/B + Trusted Types	Developer-integrated	Asserted	N/A	Yes, within CSP enforcement	Whatever the registered policy covers
C alone (2026-03-29 draft)	Extension, no cooperation	Inferred (single-stage classifier)	No — gap named by D	Partial — timing treated as assumed	63 wrapped APIs
D (2026-03-30 revision)	Extension, no cooperation	Inferred (bootstrap + selective)	Yes	Partial — timing measured, not assumed; malicious-origin case out of scope	Full prototype-level surface
This document (A+B+C+D unified)	Both, composable	Asserted, outranking inferred where both exist	Yes (L6 Stage A)	Strongest available — Trusted Types pairing plus measured extension timing	Full surface, both deployment paths

8. Data Flow Specification for the Architecture Diagram

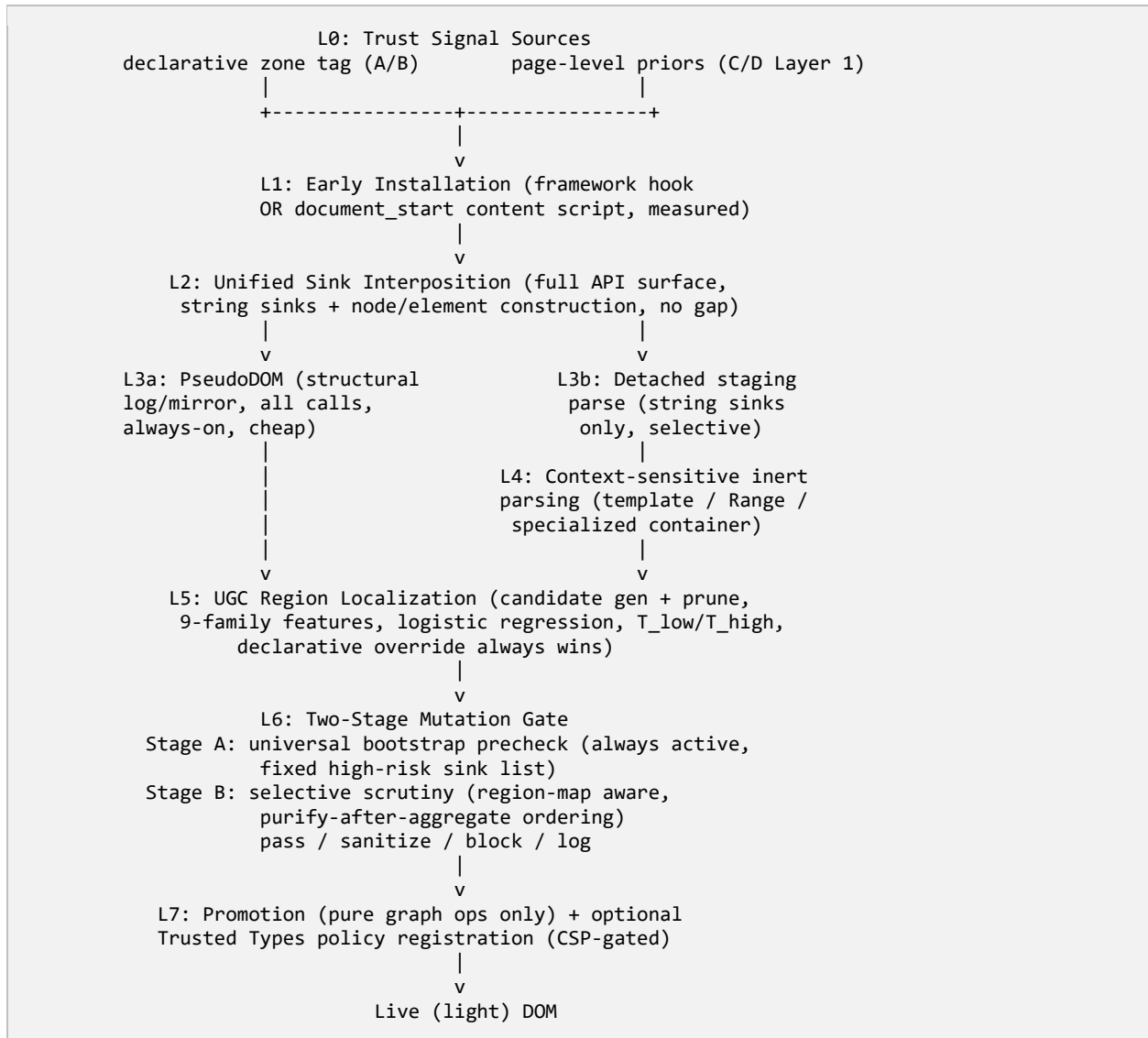


Figure 1. The full reconciled pipeline. L3a/L3b run in parallel per mutation; L4 only fires for the subset L3b receives (string sinks flagged by L5 for scrutiny).

9. Provenance Map

Which document contributed which architectural element, and how this document changed it.

Element	Originating source	Change made here
Detached-template staging + IN_PLACE purification	B	Retained; now feeds from L4's context-sensitive parse instead of a single generic template

Element	Originating source	Change made here
Batch-then-purify ordering (fusion-bypass prevention)	B	Retained unchanged, now scoped to L6 Stage B
Tier 1 / Tier 2 call split	B	Retired — replaced by C/D's full prototype-level coverage, which never had the live-node gap the split was working around
Attribute/URL scheme-allowlist lane	B	Retained as a dedicated lane, independent of the parse pipeline
Host-config / custom-renderer hooking	B	Retained as the declarative branch's L1 installation path
63-API wrapped sink surface	C	Retained; merged with B's attribute lane and D's corrections
Wildcard-XPath candidate generation	C	Retained; D's explicit pruning step added
Logistic regression UGC classifier	C	Retained with D's justification (speed, interpretability, small-dataset generalization) and declarative override added
Single-stage mutation gate	C	Replaced by D's two-stage bootstrap + selective gate
Generic detached-document parsing for string sinks	C (and B, independently)	Replaced by D's context-sensitive inert parsing (L4)
Trusted Types as a headline enforcement layer	C	Downgraded to optional hardening, matching D's independent correction and B's own §3.6
Two deployment modes (extension vs. measurement)	D	Adopted directly; declarative branch given an equivalent honesty split in §5
Bypass 1–7 taxonomy	D	Merged with B's gap analysis into the §4 table
Candidate pruning, three-state region map, T_low/T_high calibration	D	Adopted directly into L5
Ethics and privacy scope	D	Adopted directly into §6

10. Summary and Recommendation

The four sources were never actually in conflict once the deployment-model distinction in §2 is made explicit. B solved the problem for a cooperating developer and, in doing so, left a live-node gap that C/D's full prototype-level coverage closes for free. C/D solved the problem for an uncooperative site and, in doing so, had to build inference machinery (candidate generation, a classifier, calibrated thresholds) that a cooperating developer never needs, because the same information exists in the declarative signal at zero cost. D's own revision of C — the bootstrap gate, context-sensitive parsing, measured rather than assumed timing — improves both branches simultaneously, since neither A/B nor the original C had an answer for the SPA bootstrap gap or the parser-context mismatch.

The combined architecture in §3 is not a compromise between four designs; it is the version each individual document was already reaching for, assembled from the parts that were each already correct.

10.1 Open Items Before Implementation

- Calibrate `T_low`/`T_high` on a held-out domain split once real candidate labels exist (D's own stated blocker).
- Measure L1 installation timing empirically across the microbenchmark suite D specifies (inline/deferred/async/module/dynamic-import/frame-created scripts) rather than treating either path as guaranteed.
- Decide the DOM-clobbering purification default (Appendix C) explicitly rather than inheriting whatever a copied sanitizer config happens to set.
- Confirm Appendix A's sink list stays current as new browser APIs ship — same caveat as B's original Appendix A.

Appendix A — Unified Sink Coverage (L2)

Family	APIs	Priority
Node mutation methods	appendChild, insertBefore, replaceChild, removeChild	P0
Element mutation methods	append, prepend, before, after, replaceWith, replaceChildren, insertAdjacentElement	P0
String-based sinks	innerHTML, outerHTML, insertAdjacentHTML, document.write, document.writeln	P0
Attribute setters	setAttribute, setAttributeNS, removeAttribute (dangerous-name subset per Appendix C)	P0
Node/element construction	createElement, createElementNS, createTextNode, createDocumentFragment, importNode, adoptNode	P1
Range-based fragment creation	Range#createContextualFragment	P1
Shadow DOM	attachShadow	P1
Element-specific property sinks	HTMLScriptElement.text, HTMLIFrameElement.srcdoc, and similar getter/setter pairs	P1
Structural tracking only	Additional read-oriented or low-risk mutation APIs instrumented for PseudoDOM accuracy without active blocking	P2

Appendix B — UGC Feature Families (L5)

Family	Examples
Structural repetition	Dominant sibling-family size, homogeneity ratio, minimum-K threshold passes, reply-nesting depth
Semantic keywords	comment/reply/thread/review/forum/discussion/answer/feedback hits across tag, class, id, attributes, and first direct text; ARIA and schema.org markers
Identity coverage	Author element, avatar, profile-link coverage as a ratio over the dominant sibling family
Time signals	Relative and absolute time text, <time datetime> presence, timestamp attribute forms
Per-unit actions	Reply, react, share, report, edit/delete, vote, permalink coverage
Composer proximity	Sibling distance to nearest textarea/contenteditable, submit-button presence, pagination adjacency
Text statistics	Word count, link density, external-link proportion, empty-unit proportion
Negative controls	Price/currency text, add-to-cart presence, navigation ancestry, table-row structure, star-rating-only patterns

Family	Examples
Page-global priors (L0)	CSP posture, Open Graph type, JSON-LD comment hints, framework fingerprint, comment-route hints

Appendix C — Purification Checklist Additions

Item	Note
on* attributes	Reject unconditionally regardless of source (B, retained)
href / src / action / formaction / xlink:href	Scheme allowlist; reject javascript: (B §3.4, retained)
style	Reject expression(), behavior:, disallowed url() schemes (B, retained)
id / name collisions with global identifiers	DOM clobbering — requires named-property sanitization explicitly enabled, not assumed default (added from C's Table 8 evaluation)